

3.1.1 Lists (Ordered, Mutable, Methods)

فہرست (List) مربع بریکٹ [] میں لکھی جاتی ہے، ترتیب برقرار رکھتی ہے اور تبدیل کی جا سکتی ہے (Mutable)۔

CS XII

Chapter 3: Programming Fundamentals

Lecture 02 of 18

★ Golden

3.1.1 Lists (Ordered, Mutable, Methods) ★ Golden

LEARN IT

A **list** is Python's most versatile built-in data structure. It stores an **ordered** collection of items in a single variable. Lists are **mutable** (you can change them after you create them) and they are written with **square brackets** [].

Why use a list? One variable can hold thousands of values. You can mix types if you need to. You can add, remove and sort items as the program runs.

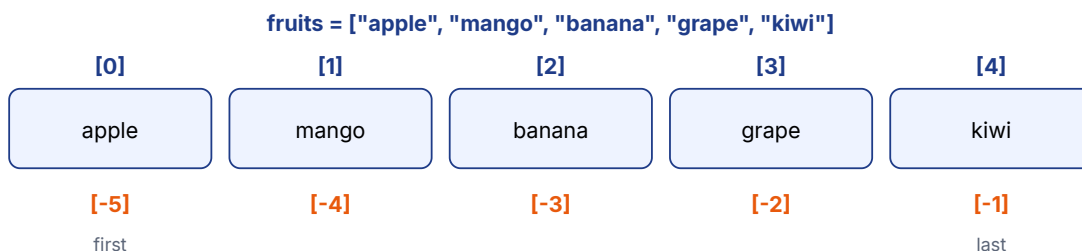
```
numbers = [10, 2, 15, 14, 3, 33]      # list of integers
names = ["Hassan", "Qasim", "Malika"] # list of strings
mixed = [1, "Python", 3.14, True]    # mixed data types
print(numbers)
print(names)
print(mixed)
```

OUTPUT

```
[10, 2, 15, 14, 3, 33]
['Hassan', 'Qasim', 'Malika']
[1, 'Python', 3.14, True]
```

List indexing (positive and negative)

Each item sits in a numbered slot called an **index**. Numbering starts at **0** from the left. Python also supports **negative indexing** from the right: **-1** is the last item, **-2** is the second last, and so on.



Positive indices count left to right from 0. Negative indices count right to left from -1.

```
fruits = ["apple", "mango", "banana", "grape", "kiwi"]
print(fruits[0])    # first item
print(fruits[4])    # last item (when you know the length)
print(fruits[-1])   # last item (works even if the list grows)
print(fruits[-5])   # first item, counted from the right
```

OUTPUT

```
apple
kiwi
kiwi
apple
```

REMEMBER IT

Always remember `[-1]`. If a list has 10,000 items and you do not want to compute `len(list) - 1`, write `list[-1]`. It grabs the last item in one step.

Slicing a list

Slicing copies a portion of a list into a **new** list. The original is not changed.

Syntax: `list[start:stop:step]`

- `start` — first index to include (default 0).
- `stop` — first index to **exclude**. The slice goes up to, but does **not** include, this index.
- `step` — jump size (default 1). A step of `-1` walks backwards.

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(nums[2:6])
print(nums[:4])
print(nums[6:])
print(nums[::2])
print(nums[::-1])
```

OUTPUT

```
[2, 3, 4, 5]
[0, 1, 2, 3]
[6, 7, 8, 9]
[0, 2, 4, 6, 8]
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Trace of `nums[2:6]`: start at index 2 (value 2), take 3, 4, 5, and stop **before** index 6. Result `[2, 3, 4, 5]`. `nums[::-1]` is the standard way to reverse a list without changing the original (it returns a new reversed list).

Traversal

Traversal means visiting each element, one by one, in order. Use a `for` loop (the usual Python style) or a `while` loop with an index.

```
numsList = [10, 20, 30]
for num in numsList:
    print(num)
```

OUTPUT

```
10
20
30
```

```
numsList = [10, 20, 30]
index = 0
while index < len(numsList):
    print(numsList[index])
    index += 1
```

OUTPUT

```
10
20
30
```

The `for` loop gives you each value directly. The `while` loop needs a counter, a test `index < len(numsList)`, and `index += 1` so the loop ends.

Built-in list methods

Because a list is mutable, Python gives you methods that change it in place. `len()` is listed with them in many notes, but it is a **built-in function**: you write `len(fruits)`, not `fruits.len()`.

Operation	What it does
<code>len(list)</code>	Returns the number of items.
<code>.append(value)</code>	Adds an item at the end .
<code>.insert(index, value)</code>	Inserts an item at a chosen position (later items shift right).
<code>.remove(value)</code>	Removes the first matching value. Raises <code>ValueError</code> if it is missing.
<code>.clear()</code>	Removes every item. The list becomes <code>[]</code> .
<code>.count(value)</code>	How many times a value appears.
<code>.index(value)</code>	Index of the first occurrence.
<code>.sort()</code>	Sorts the list in place, ascending. Use <code>reverse=True</code> for descending.
<code>.reverse()</code>	Reverses the list in place (does not sort).

WORKED EXAMPLE

Worked example: methods change the same list step by step.

```
fruits = ['apple', 'banana', 'apple']
print(fruits)
fruits.append('grape')
print(fruits)
fruits.remove('apple')
print(fruits)
fruits.sort()
print(fruits)
```

OUTPUT

```
['apple', 'banana', 'apple']
['apple', 'banana', 'apple', 'grape']
['banana', 'apple', 'grape']
['apple', 'banana', 'grape']
```

After this line	List in memory
create	<code>['apple', 'banana', 'apple']</code>
<code>.append('grape')</code>	<code>['apple', 'banana', 'apple', 'grape']</code>
<code>.remove('apple')</code>	<code>['banana', 'apple', 'grape']</code> — only the first apple goes
<code>.sort()</code>	<code>['apple', 'banana', 'grape']</code>

More methods on numbers:

```

nums = [10, 20, 30]
print(len(nums))
nums.insert(1, 15)
print(nums)
print(nums.count(20))
print(nums.index(30))
nums.reverse()
print(nums)
nums.clear()
print(nums)

```

OUTPUT

```

3
[10, 15, 20, 30]
1
3
[30, 20, 15, 10]
[]

```

`insert(1, 15)` places 15 at index 1, so 20 and 30 shift right. After that, 30 sits at index 3. `reverse()` flips the current order; it does not sort. `clear()` leaves an empty list `[]`, not the value `None`.

You can also replace an item by index, because lists are mutable:

```

numbers = [10, 2, 15]
numbers[0] = 99
print(numbers)

```

OUTPUT

```

[99, 2, 15]

```

Advantages of lists

- **Mutable** — add, remove or replace items as the program runs.
- **Organised** — thousands of values live in one variable instead of `n1`, `n2`, `n3`...
- **Flexible** — integers, strings, booleans and even other lists can sit in the same list.

Activity: searching an element in a list

This program combines a list, user input, a `for` loop, an `if` test and a **flag** (a Boolean that remembers whether the search succeeded).

```

numbers = [5, 10, 15, 20, 25]
search = int(input("Enter number to search: "))
found = False
for num in numbers:
    if num == search:
        found = True
        break
if found:
    print("Number found in the list.")
else:
    print("Number NOT found in the list.")

```

Execution 1 — the user types 15, which is at index 2. The loop sets `found = True` and `break` stops the scan at once.

OUTPUT

```
Enter number to search: 15
Number found in the list.
```

Execution 2 — the user types 99. The loop finishes without a match, so the flag stays `False`.

OUTPUT

```
Enter number to search: 99
Number NOT found in the list.
```

Why `break`? If the value is found at index 1 in a list of 10,000 items, there is no need to check the remaining 9,998. `break` makes linear search cheaper as soon as it succeeds.

REMEMBER IT

LIST — Lots of items, Indexed from 0, Square brackets, Tweakable (mutable). Slice memory trick: "start is in, stop is out."

COMMON MISTAKE

Mistake 1: `.remove('apple')` deletes every apple. **Fix:** it deletes only the first match. Call it again if you need the next one, or use a loop.

Mistake 2: `.sort()` and `sorted()` are the same. **Fix:** `.sort()` is a list method that changes the original and returns `None`. `sorted()` is a function that works on many structures and returns a **new** list (see 3.1.5).

Mistake 3: `fruits[5]` on a five-item list. Valid positive indices are 0 to 4. Use `[-1]` for the last item.

اردو میں سمجھیں

فہرست (List) مربع بریکٹ `[]` میں لکھی جاتی ہے، ترتیب برقرار رکھتی ہے اور تبدیل کی جا سکتی ہے (Mutable). انڈیکس صفر سے شروع ہوتا ہے، اور `[-1]` آخری عنصر دیتا ہے۔ `append()` آخر میں جوڑتا ہے، `remove()` پہلی مشابہ قیمت ہٹاتا ہے، اور `sort()` اصل فہرست کو ترتیب دیتا ہے۔ تلاش کرتے وقت `break` مل جانے کے بعد لوپ روک دیتا ہے۔

EXAM TIP

Lists appear every year: "What is a list?", "List four methods with one line each", "Differentiate list and tuple", "Write a program to search an element", "Predict the output of a slice." Write: ordered, mutable, `[]`, index from 0, `[-1]` for last. For a slice, always say the stop index is excluded. For `.remove()`, say "first occurrence only." Keep a short program ready that uses `append`, `remove` and `sort` with the printed list after each line.

CHECK YOURSELF

1. What does `nums[2:6]` return if `nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`?
2. After `fruits = ['apple', 'banana', 'apple']` then `fruits.remove('apple')`, what is `fruits`?
3. Why does the search program use `break`?

Answers: 1. `[2, 3, 4, 5]`. 2. `['banana', 'apple']` (only the first apple is removed). 3. To stop scanning once the value is found, so the rest of a long list is not checked.

Next lecture: 3.1.2 Tuples (Ordered, Immutable)

